

Торіс 1. Параллельная редукция массивов на CPU и GPU

Supervisor: Assistant Professor Sadvakasova Kuralay

Course: Heterogeneous Parallelization

Team members: 1. Zhanibek Meiramkan ADA-2404M

2. Ayazhan Aitzhanova ADA-2403M

3. Diana Kim ADA-2403M



Цель работы

Реализовать и сравнить вычисление суммы и максимума одномерного массива:

- последовательно на CPU,
- параллельно на CPU с OpenMP,
- параллельно на GPU с CUDA,

и проанализировать:

- время выполнения,
- корректность результатов,
- ускорение (speedup).



Постановка задачи

Редукция

В данной работе рассматривается задача редукции одномерного массива целых чисел с вычислением:

- суммы элементов массива (sum)
- максимального элемента массива (max)

Исходные данные — одномерный массив целых чисел.

Редукция — это операция, которая объединяет множество элементов в одно значение. Например, вычисление суммы, минимума или максимума массива.

Мы реализовали три версии одной и той же задачи:

- **Sequential CPU** — базовая версия. Используется как эталон корректности и базовое время.
- **OpenMP CPU** — параллелизация на многоядерном CPU. Позволяет оценить ускорение за счёт потоков.
- **CUDA GPU** — массово-параллельная реализация. Позволяет показать преимущества GPU для операций редукции.

Генерация данных и условия эксперимента

Входные данные

- массив заполняется случайными числами `rand() % 1000`,

```
int main() {  
    srand(123);                // фиксируем seed, чтобы результаты были повторяемыми
```

- используется фиксированный seed `srand(123)` для воспроизводимости,

```
static void run_size(int n) {  
    vector<int> a(n);           // запускаем тест на одном размере массива  
    for (int i = 0; i < n; i++) a[i] = rand() % 1000; // создаём массив на CPU из n элементов  
                                // заполняем случайными числами
```

- размеры массивов:
 - 100 000
 - 1 000 000
 - 10 000 000 элементов.

```
run_size(100000);              // тест для 100k элементов  
run_size(1000000);             // тест для 1 млн элементов  
run_size(10000000);            // тест для 10 млн элементов
```

Последовательная версия

Последовательная версия выполняется на одном потоке CPU:

- используется обычный цикл for,
- вычисляются сумма и максимум массива.

```
static long long cpu_sum_seq(const int* a, int n) {           // последовательная сумма массива
    long long s = 0;                                         // это аккумулятор суммы, чтобы не переполнить int
    for (int i = 0; i < n; i++) s += a[i];                  // проходим по всем элементам и суммируем
    return s;                                                 // возвращаем итоговую сумму
}

static int cpu_max_seq(const int* a, int n) {                // последовательный максимум массива
    int m = INT_MIN;                                         // начинаем с минимального int
    for (int i = 0; i < n; i++)                             // проходим по всем элементам массива
        if (a[i] > m) m = a[i];                             // обновляем максимум, если нашли больше
    return m;                                                 // возвращаем найденный максимум
}
```

```
auto s1 = chrono::high_resolution_clock::now();             // старт таймера для sequential CPU
long long sum_seq = cpu_sum_seq(a.data(), n);               // считаем сумму последовательно
int max_seq = cpu_max_seq(a.data(), n);                     // считаем максимум последовательно
auto e1 = chrono::high_resolution_clock::now();             // конец таймера sequential CPU
double t_seq = chrono::duration<double, milli>(e1 - s1).count(); // переводим длительность в миллисекунды
```

*Это baseline для speedup

Параллельная версия с OpenMP

Для распараллеливания используется директива:

```
static long long cpu_sum_omp(const int* a, int n) {           // сумма на CPU с OpenMP
    long long s = 0;                                         // общий аккумулятор суммы
    #pragma omp parallel for reduction(+:s)                 // делим цикл по потокам и делаем reduction суммы
    for (int i = 0; i < n; i++) s += a[i];                  // каждый поток добавляет свою часть элементов
    return s;                                                 // возвращаем объединённую сумму
}

static int cpu_max_omp(const int* a, int n) {                // максимум на CPU с OpenMP
    int m = INT_MIN;                                         // общий максимум (начинаем с минимума)
    #pragma omp parallel for reduction(max:m)               // делим цикл по потокам и делаем reduction(max)
    for (int i = 0; i < n; i++)                             // параллельный проход по индексам
        if (a[i] > m) m = a[i];                             // каждый поток обновляет свой локальный максимум
    return m;                                                 // возвращаем глобальный максимум после reduction
}
```

- цикл автоматически делится между потоками CPU,
- каждый поток считает частичный результат,
- reduction объединяет результаты потоков.

Замер времени OpenMP + speedup

```
auto s2 = chrono::high_resolution_clock::now();             // старт таймера для OpenMP CPU
long long sum_omp = cpu_sum_omp(a.data(), n);                // считаем сумму с OpenMP
int max_omp = cpu_max_omp(a.data(), n);                      // считаем максимум с OpenMP
auto e2 = chrono::high_resolution_clock::now();              // конец таймера OpenMP CPU
double t_omp = chrono::duration<double, milli>(e2 - s2).count(); // длительность OpenMP в миллисекундах
```

*Это baseline для speedup

Ограничения OpenMP

Почему ускорение ограничено

- количество потоков ограничено числом ядер CPU,
- накладные расходы на синхронизацию,
- ускорение значительно меньше, чем на GPU.

OpenMP эффективно ускоряет вычисления, но масштабируемость ограничена архитектурой CPU.



CUDA и идея GPU-редукции

Почему GPU подходит для редукции

Графический процессор (GPU) содержит тысячи аппаратных потоков, что позволяет эффективно выполнять операции, которые:

- применяются ко всем элементам массива,
- не зависят друг от друга на начальном этапе.

Редукция (sum, max) является типичной задачей data-parallel вычислений, поэтому хорошо масштабируется на GPU.

1



Организация вычислений в CUDA

В CUDA вычисления организованы иерархически:

- Thread (поток)
- Обработывает один элемент массива.
- Block (блок)
- Группа потоков, которая:
 - загружает данные в shared memory,
 - вычисляет частичную сумму и максимум.
- Grid (сетка блоков)
- Покрывает весь массив целиком.

2



Использование shared memory

Внутри каждого блока применяется shared memory — быстрая память, общая для всех потоков блока.

Она используется для:

- хранения промежуточных значений суммы;
- хранения промежуточных значений максимума;
- выполнения параллельной редукции внутри блока.

3



Параллельная редукция (tree-based reduction)

Редукция выполняется по схеме двоичного дерева:

- на каждом шаге число активных потоков уменьшается в 2 раза;
- шаг редукции задаётся как $\text{stride} \gg= 1$;
- используется синхронизация `__syncthreads()`.

В результате каждый блок вычисляет:

- одну частичную сумму;
- один частичный максимум.

Финальное объединение результатов выполняется на CPU.

CUDA kernel (shared memory + syncthreads)

```

__global__ void reduce_sum_max_kernel(const int* a, int n,
                                     unsigned long long* block_sums,
                                     int* block_max) {
    extern __shared__ unsigned char shmem[];
    unsigned long long* sh_sum = (unsigned long long*)shmem;
    int* sh_max = (int*)(sh_sum + blockDim.x);
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + tid;
    unsigned long long v_sum = 0ULL;
    int v_max = INT_MIN;
    if (gid < n) {
        int x = a[gid];
        v_sum = (unsigned long long)x;
        v_max = x;
    }

    sh_sum[tid] = v_sum;
    sh_max[tid] = v_max;
    __syncthreads();
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sh_sum[tid] += sh_sum[tid + stride];
            int other = sh_max[tid + stride];
            if (other > sh_max[tid]) sh_max[tid] = other;
        }
        __syncthreads();
    }

    if (tid == 0) {
        block_sums[blockIdx.x] = sh_sum[0];
        block_max[blockIdx.x] = sh_max[0];
    }
}

```

В kernel:

- данные загружаются в shared memory,
- выполняется дерево редукции,
- шаг редукции уменьшается в 2 раза (stride >>= 1).

Используется:

- __syncthreads() для синхронизации потоков,
- shared memory для ускорения доступа к данным.

Host-код CUDA: память, kernel, замеры времени

```
static void gpu_sum_max(const int* h, int n,
    unsigned long long& sum_out,
    int& max_out,
    float& gpu_kernel_ms_out,
    float& gpu_total_ms_out) {
    // функция считает sum+max на GPU для массива на CPU
    // выходной параметр: итоговая сумма
    // выходной параметр: итоговый максимум
    // выход: время только kernel (мс)
    // выход: время H2D+kernel+D2H (мс)

    int* d_a = nullptr;
    cuda_ok(cudaMalloc(&d_a, (size_t)n * sizeof(int)), "malloc d_a");
    // указатель на массив в памяти GPU
    // выделяем память на GPU под входной массив
    int threads = 256;
    int blocks = (n + threads - 1) / threads;
    // число потоков в одном блоке
    // число блоков: округление вверх

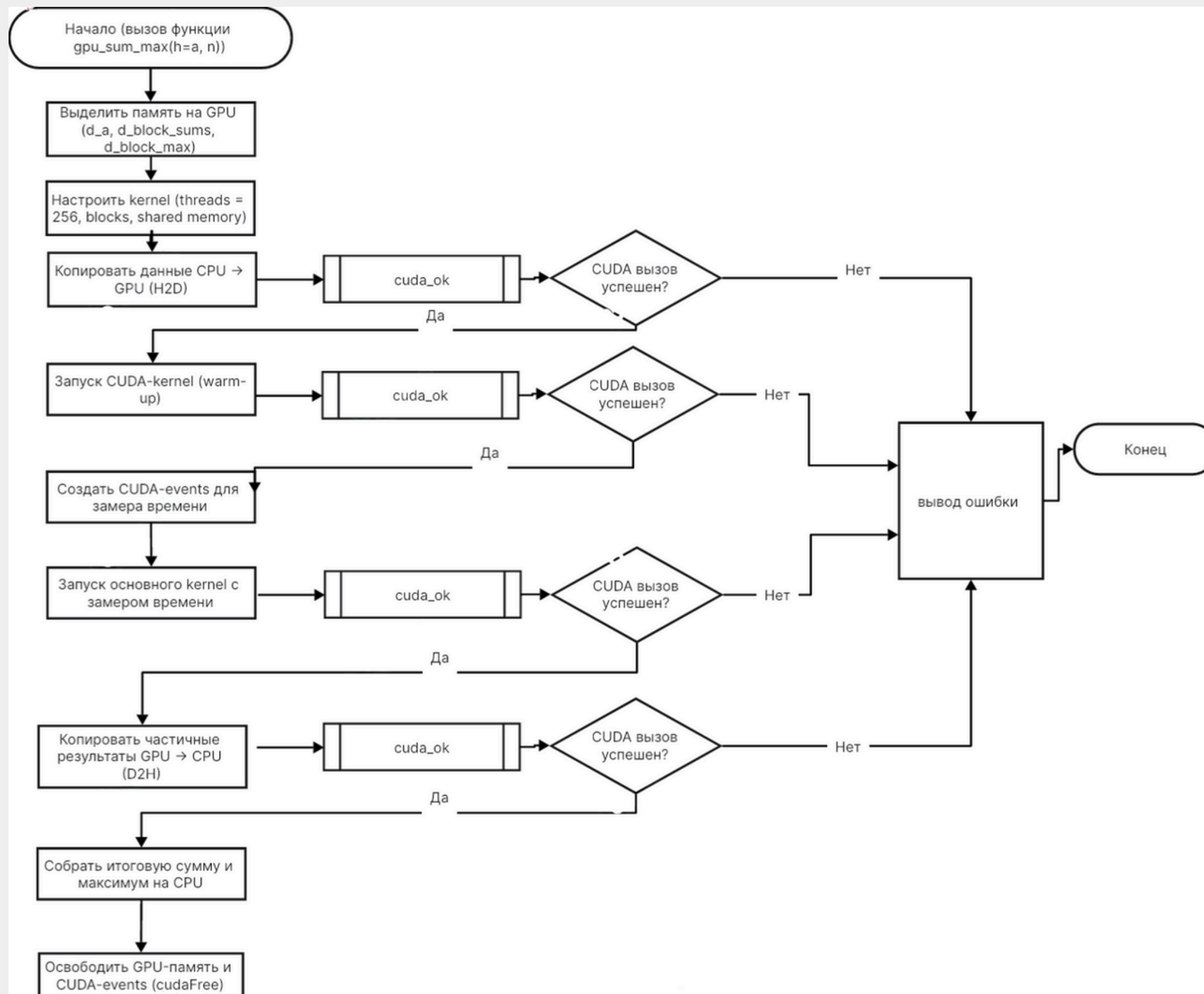
    unsigned long long* d_block_sums = nullptr;
    int* d_block_max = nullptr;
    // массив частичных сумм блоков на GPU
    // массив частичных максимумов блоков на GPU
    cuda_ok(cudaMalloc(&d_block_sums, (size_t)blocks * sizeof(unsigned long long)), // выделяем память под суммы блоков
        "malloc d_block_sums");
    // подпись для сообщения об ошибке
    cuda_ok(cudaMalloc(&d_block_max, (size_t)blocks * sizeof(int)), // выделяем память под максимумы блоков
        "malloc d_block_max");
    // подпись для сообщения об ошибке

    size_t sh_bytes = (size_t)threads * sizeof(unsigned long long) // shared memory под сумму (по потоку)
        + (size_t)threads * sizeof(int); // shared memory под максимум (по потоку)

    cuda_ok(cudaMemcpy(d_a, h, (size_t)n * sizeof(int), cudaMemcpyHostToDevice), "copy h2d warmup");
    // копируем входные данные на GPU
    // направление копирования H2D
    reduce_sum_max_kernel<<<blocks, threads, sh_bytes>>>(&d_a, n, // запускаем kernel (прогревочный запуск)
        d_block_sums, d_block_max); // куда писать частичные результаты
    cuda_ok(cudaGetLastError(), "kernel warmup"); // проверяем, что kernel запустился без ошибок
    cuda_ok(cudaDeviceSynchronize(), "sync warmup"); // ждём завершения warm-up kernel
```

Блок-схема функции `gri_sum_max`

`gri_sum_max` выполняет подсчет суммы и максимального элемента массива на GPU для массива на CPU



```

cudaEvent_t k_start, k_stop, t_start, t_stop;
cuda_ok(cudaEventCreate(&k_start), "create k_start");
cuda_ok(cudaEventCreate(&k_stop), "create k_stop");
cuda_ok(cudaEventCreate(&t_start), "create t_start");
cuda_ok(cudaEventCreate(&t_stop), "create t_stop");

cuda_ok(cudaEventRecord(t_start), "record total start");
cuda_ok(cudaMemcpy(d_a, h, (size_t)n * sizeof(int),
    cudaMemcpyHostToDevice), "copy h2d");

cuda_ok(cudaEventRecord(k_start), "record kernel start");
reduce_sum_max_kernel<<<blocks, threads, sh_bytes>>>(d_a, n,
    d_block_sums, d_block_max);
cuda_ok(cudaGetLastError(), "kernel launch");
cuda_ok(cudaEventRecord(k_stop), "record kernel stop");
cuda_ok(cudaEventSynchronize(k_stop), "sync kernel stop");
cuda_ok(cudaEventElapsedTime(&gpu_kernel_ms_out, k_start, k_stop),
    "elapsed kernel");

vector<unsigned long long> h_block_sums(blocks);
vector<int> h_block_max(blocks);

cuda_ok(cudaMemcpy(h_block_sums.data(), d_block_sums,
    (size_t)blocks * sizeof(unsigned long long),
    cudaMemcpyDeviceToHost), "copy sums d2h");
cuda_ok(cudaMemcpy(h_block_max.data(), d_block_max,
    (size_t)blocks * sizeof(int),
    cudaMemcpyDeviceToHost), "copy max d2h");

cuda_ok(cudaEventRecord(t_stop), "record total stop");
cuda_ok(cudaEventSynchronize(t_stop), "sync total stop");
cuda_ok(cudaEventElapsedTime(&gpu_total_ms_out, t_start, t_stop),
    "elapsed total");

```

```

// CUDA события для измерения времени
// создаём событие начала kernel-таймера
// создаём событие конца kernel-таймера
// создаём событие начала total-таймера
// создаём событие конца total-таймера

```

```

// старт total time (копирование+kernel+копирование)
// копируем массив на GPU для реального замера
// направление H2D

```

```

// старт замера времени kernel
// запускаем основной kernel
// массивы частичных результатов
// проверяем ошибки запуска kernel
// ставим событие конца kernel
// ждём, пока kernel точно завершится
// считаем время kernel в миллисекундах
// подпись для проверки ошибок

```

```

// буфер на CPU для сумм блоков (после D2H)
// буфер на CPU для максимумов блоков (после D2H)

```

```

// копируем суммы блоков с GPU на CPU
// направление D2H
// копируем максимумы блоков с GPU на CPU
// направление D2H

```

```

// ставим событие окончания total time
// ждём завершения всех операций до t_stop
// считаем total время (H2D+K+D2H)
// подпись для проверки ошибок

```

```

unsigned long long s = 0ULL;
int m = INT_MIN;
for (int i = 0; i < blocks; i++) {
    s += h_block_sums[i];
    if (h_block_max[i] > m) m = h_block_max[i];
}

```

```

// итоговая сумма (после объединения блоков)
// итоговый максимум (после объединения блоков)
// проходим по всем блокам (их мало по сравнению с n)
// складываем частичные суммы блоков
// берём максимум среди максимумов блоков

```

```

sum_out = s;
max_out = m;

```

```

// записываем итоговую сумму в выходной параметр
// записываем итоговый максимум в выходной параметр

```

```

cuda_ok(cudaEventDestroy(k_start), "destroy k_start");
cuda_ok(cudaEventDestroy(k_stop), "destroy k_stop");
cuda_ok(cudaEventDestroy(t_start), "destroy t_start");
cuda_ok(cudaEventDestroy(t_stop), "destroy t_stop");

```

```

// удаляем событие начала kernel
// удаляем событие конца kernel
// удаляем событие начала total
// удаляем событие конца total

```

```

cuda_ok(cudaFree(d_a), "free d_a");
cuda_ok(cudaFree(d_block_sums), "free d_block_sums");
cuda_ok(cudaFree(d_block_max), "free d_block_max");

```

```

// освобождаем память входного массива на GPU
// освобождаем память частичных сумм на GPU
// освобождаем память частичных максимумов на GPU

```

Эксперимент, сравнение результатов

```
static void run_size(int n) {  
    vector<int> a(n);  
    for (int i = 0; i < n; i++) a[i] = rand() % 1000;  
  
    auto s1 = chrono::high_resolution_clock::now();  
    long long sum_seq = cpu_sum_seq(a.data(), n);  
    int max_seq = cpu_max_seq(a.data(), n);  
    auto e1 = chrono::high_resolution_clock::now();  
    double t_seq = chrono::duration<double, milli>(e1 - s1).count();  
  
    auto s2 = chrono::high_resolution_clock::now();  
    long long sum_omp = cpu_sum_omp(a.data(), n);  
    int max_omp = cpu_max_omp(a.data(), n);  
    auto e2 = chrono::high_resolution_clock::now();  
    double t_omp = chrono::duration<double, milli>(e2 - s2).count();  
  
    unsigned long long sum_gpu = 0ULL;  
    int max_gpu = 0;  
    float t_gpu_kernel = 0.0f;  
    float t_gpu_total = 0.0f;  
    gpu_sum_max(a.data(), n, sum_gpu, max_gpu,  
                t_gpu_kernel, t_gpu_total);  
  
    bool ok_sum = (sum_seq == (long long)sum_gpu)  
                && (sum_omp == (long long)sum_gpu);  
    bool ok_max = (max_seq == max_gpu)  
                && (max_omp == max_gpu);  
  
    double speed_omp = (t_omp > 0.0 ? t_seq / t_omp : 0.0);  
    double speed_gpu_kernel = (t_gpu_kernel > 0.0 ? t_seq / (double)t_gpu_kernel : 0.0);  
    double speed_gpu_total = (t_gpu_total > 0.0 ? t_seq / (double)t_gpu_total : 0.0);  
}
```

// запускаем тест на одном размере массива
// создаём массив на CPU из n элементов
// заполняем случайными числами

// старт таймера для sequential CPU
// считаем сумму последовательно
// считаем максимум последовательно
// конец таймера sequential CPU
// переводим длительность в миллисекунды

// старт таймера для OpenMP CPU
// считаем сумму с OpenMP
// считаем максимум с OpenMP
// конец таймера OpenMP CPU
// длительность OpenMP в миллисекундах

// здесь будет сумма с GPU
// здесь будет максимум с GPU
// время только kernel (мс)
// время H2D+kernel+D2H (мс)
// запускаем GPU версию и получаем результаты
// получаем оба времени замера

// проверяем: сумма GPU совпала с seq
// проверяем: сумма OpenMP тоже совпала
// проверяем: максимум GPU совпал с seq
// проверяем: максимум OpenMP тоже совпал

// ускорение OpenMP относительно seq
// ускорение GPU kernel
// ускорение GPU total

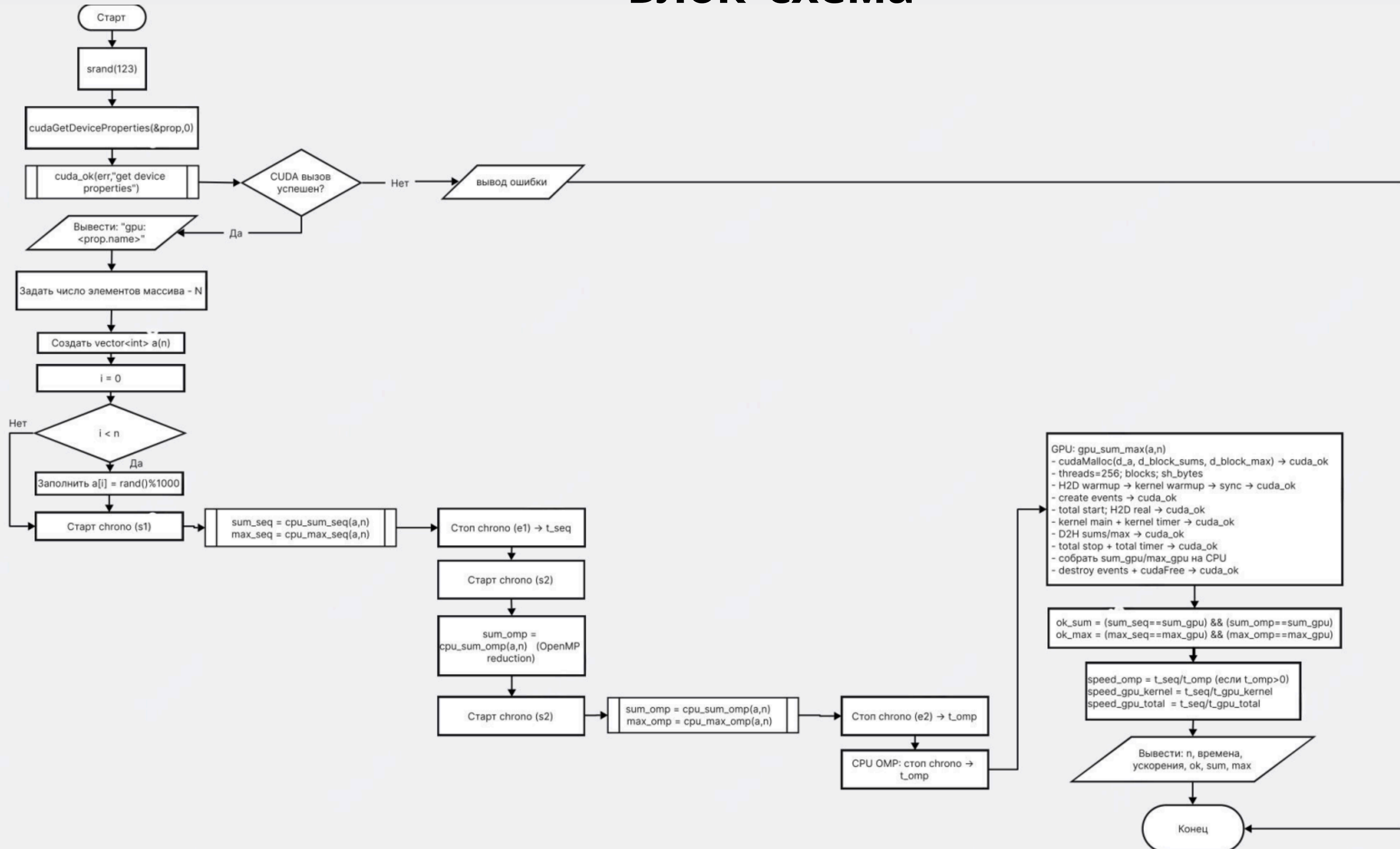
Сравнение результатов
Для каждого размера массива проверяется:

- совпадение суммы,
 - совпадение максимума,
- между:
- Sequential CPU,
 - OpenMP CPU,
 - CUDA GPU.

Если результаты совпадают — реализация считается корректной.

Ускорение показывает, насколько параллельная версия быстрее последовательной.

Блок-схема



Результат проекта

1

```
размер: 100000
seq cpu:      0.125269 мс
omp cpu:      0.185081 мс
gpu cuda kernel time: 0.024 мс
gpu cuda total: 0.196704 мс
ускорение omp:      0.676833
ускорение gpu kernel: 5.21954
ускорение gpu total: 0.63684
sum ok: да
max ok: да
sum = 49706568
max = 999
```

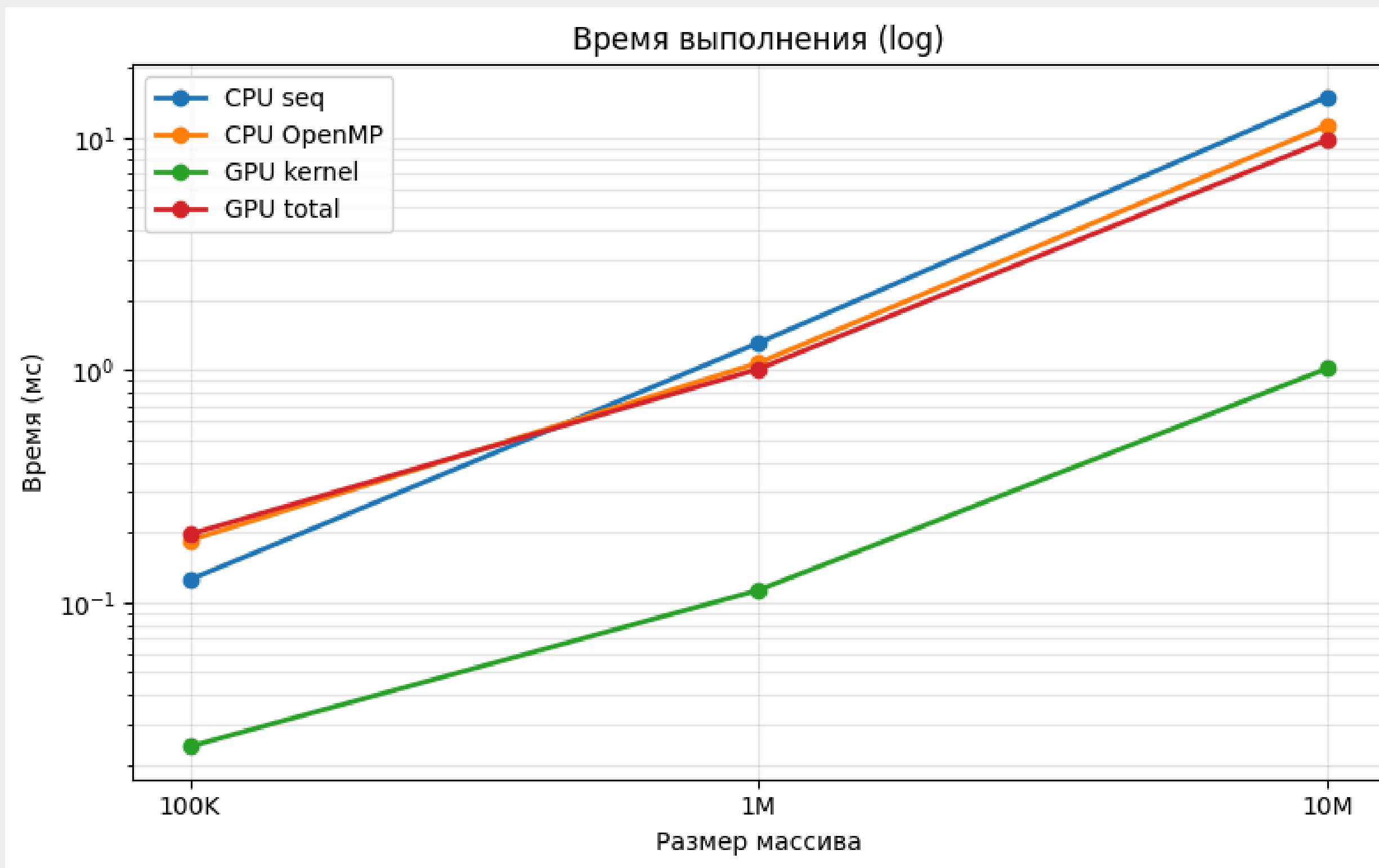
2

```
размер: 1000000
seq cpu:      1.30834 мс
omp cpu:      1.07407 мс
gpu cuda kernel time: 0.112928 мс
gpu cuda total: 1.00995 мс
ускорение omp:      1.21811
ускорение gpu kernel: 11.5856
ускорение gpu total: 1.29545
sum ok: да
max ok: да
sum = 498820231
max = 999
```

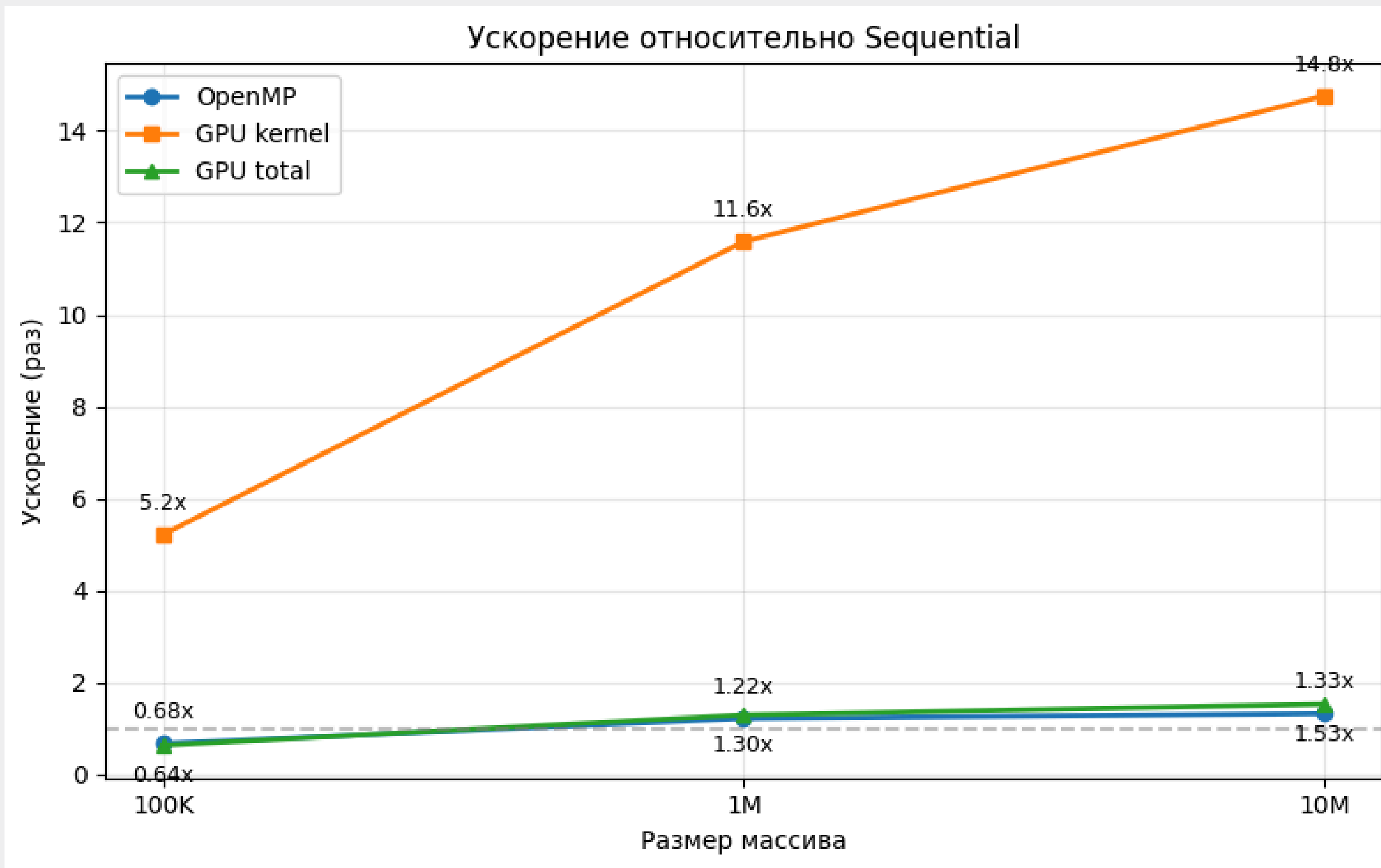
3

```
размер: 10000000
seq cpu:      14.9661 мс
omp cpu:      11.2798 мс
gpu cuda kernel time: 1.01443 мс
gpu cuda total: 9.77946 мс
ускорение omp:      1.32681
ускорение gpu kernel: 14.7532
ускорение gpu total: 1.53036
sum ok: да
max ok: да
sum = 4994438803
max = 999
```

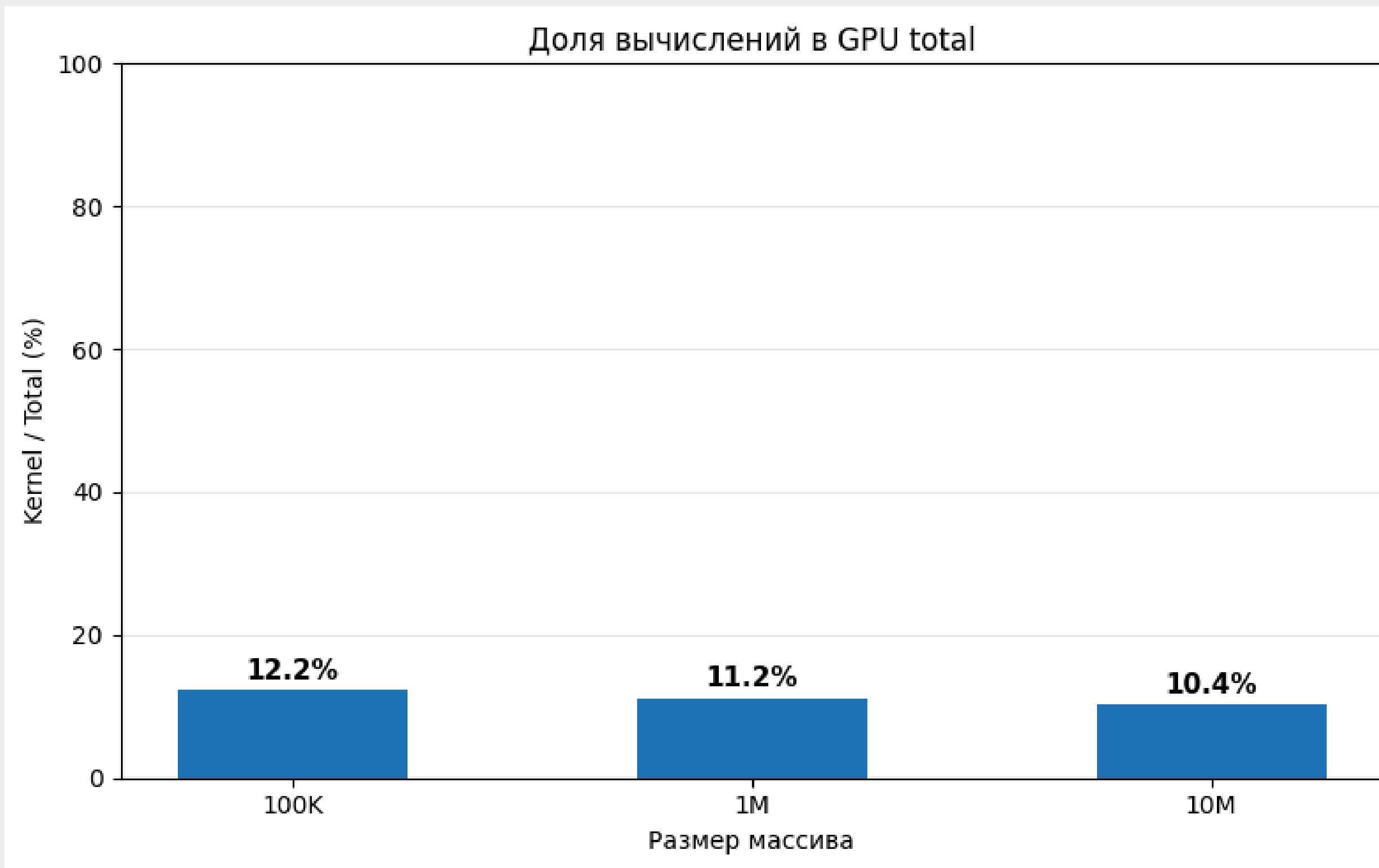
Время выполнения



Ускорение



Где теряется время GPU



Q&A

Thank You